

¿Cómo se relacionan las tablas en una base de datos MySQL?

Imagina que una base de datos es como un colegio enorme donde toda la información está organizada en diferentes carpetas.

Cada carpeta guarda un tipo de información diferente. En MySQL, esas carpetas se llaman **tablas**.

Por ejemplo:

- Una tabla guarda los **estudiantes**
- Otra tabla guarda los **cursos**
- Otra tabla guarda las **notas**

Pero aparece un problema:

¿Cómo sabe la base de datos qué estudiante pertenece a qué curso?

Ahí es donde entran las **relaciones entre tablas**.

¿Qué es una relación entre tablas?

Una relación es una conexión entre dos tablas para compartir información de forma organizada.

Es parecido a lo que pasa en la vida real:

- Un estudiante pertenece a un curso.
- Un curso tiene muchos estudiantes.
- Un estudiante puede tener muchas notas.

Las tablas “hablan” entre sí mediante algo llamado:

Claves primarias y claves foráneas

1. Clave primaria (Primary Key)

Es el identificador único de cada registro.

Por ejemplo:

Tabla **estudiantes**

id_estudiante	nombre
1	Ana
2	Carlos
3	Laura

Aquí:

- id_estudiante es la clave primaria.
- No se puede repetir.
- Identifica a cada estudiante de manera única.

Es como el número de documento de una persona.

2. Clave foránea (Foreign Key)

Es un campo que conecta una tabla con otra.

Ejemplo:

Tabla **notas**

id_nota	materia	nota	id_estudiante
1	Matemáticas	4.5	1
2	Inglés	3.8	2

El campo id_estudiante viene de la tabla estudiantes.

Entonces:

- La nota 4.5 pertenece a Ana.
- La nota 3.8 pertenece a Carlos.

La clave foránea crea la relación.

Explicación fácil de entender

Piensa en WhatsApp:

- Tú tienes un número.
- Tus amigos tienen números.

- Cuando guardas un contacto, WhatsApp relaciona nombres con números.

MySQL hace algo parecido:

- Relaciona registros usando IDs.

Tipos de relaciones

1 Uno a Uno (1:1)

Un registro se relaciona con uno solo.

Ejemplo:

- Un estudiante tiene un carnet.
- Un carnet pertenece a un solo estudiante.

2 Uno a Muchos (1:N)

Es la más utilizada.

Ejemplo:

- Un curso tiene muchos estudiantes.
- Pero cada estudiante pertenece a un solo curso.

Tabla cursos:

id_curso	nombre_curso
1	10A
2	10B

Tabla estudiantes:

id_estudiante	nombre	id_curso
1	Ana	1
2	Carlos	1
3	Laura	2

Aquí:

- Ana y Carlos pertenecen al curso 10A.
 - Laura pertenece a 10B.
-

3 Muchos a Muchos (N:M)

Muchos registros se relacionan con muchos.

Ejemplo:

- Un estudiante puede ver muchas materias.
- Una materia puede tener muchos estudiantes.

Para esto se usa una tabla intermedia.

Tabla estudiantes

id_estudiante	nombre
1	Ana

Tabla materias

id_materia	nombre
1	Matemáticas
2	Inglés

Tabla intermedia:

id_estudiante	id_materia
1	1
1	2

Esto significa:

- Ana ve Matemáticas.
 - Ana ve Inglés.
-

¿Por qué son importantes las relaciones?

Las relaciones ayudan a:

- ✓ Organizar la información
- ✓ Evitar datos repetidos
- ✓ Mantener el orden
- ✓ Buscar información más rápido
- ✓ Crear sistemas reales como:

- Facebook
 - Instagram
 - Bancos
 - Hospitales
 - Colegios
-

Ejemplo de la vida real

Piensa en Netflix:

- Hay una tabla de usuarios.
- Otra tabla de películas.
- Otra tabla de reproducciones.

Cuando ves una película, la base de datos relaciona:

Usuario + película + fecha

Gracias a eso Netflix sabe qué viste.

Ejemplo básico en MySQL

`</>` SQL

```
CREATE TABLE cursos (  
    id_curso INT PRIMARY KEY,  
    nombre_curso VARCHAR(50)  
);  
  
CREATE TABLE estudiantes (  
    id_estudiante INT PRIMARY KEY,  
    nombre VARCHAR(50),  
    id_curso INT,  
    FOREIGN KEY (id_curso) REFERENCES cursos(id_curso)  
);
```

¿Qué hace este código?

- Crea la tabla cursos.
- Crea la tabla estudiantes.
- Conecta ambas tablas usando id_curso.

Entonces MySQL entiende:

“Cada estudiante pertenece a un curso”.

Conclusión

Las relaciones en MySQL permiten conectar información entre diferentes tablas para que la base de datos funcione de manera inteligente y organizada.

Sin relaciones, las bases de datos serían un desorden enorme lleno de información repetida.

Las claves primarias identifican. Las claves foráneas conectan. Y gracias a eso podemos construir sistemas modernos y profesionales.



Ejemplo de Base de Datos de Aguacates con 4 Tablas Relacionadas

Imaginemos una empresa llamada **Aguacates del Campo** que necesita un sistema para controlar:

- Los agricultores

- Los productos
- Los clientes
- Las ventas

Todas esas partes deben conectarse entre sí para que la información tenga sentido.

Tablas del sistema

Tendremos 4 tablas:

- 1 agricultores
- 2 productos
- 3 clientes
- 4 ventas

◆ 1. Tabla Agricultores

Guarda la información de las personas que cultivan los aguacates.

id_agricultor	nombre	finca
1	Pedro Gómez	El Paraíso
2	Marta Ruiz	La Esperanza

Aquí:

- id_agricultor es la clave primaria.
- Identifica a cada agricultor.

◆ 2. Tabla Productos

Guarda los aguacates disponibles.

id_producto	nombre_producto	precio	id_agricultor
1	Aguacate Hass	5000	1
2	Aguacate Papelillo	3500	2

Aquí ocurre la primera relación:

- `id_agricultor` conecta productos con agricultores.

Esto significa:

- Pedro produce Aguacate Hass.
- Marta produce Aguacate Papelillo.

👉 Relación:

Un agricultor puede tener muchos productos.

◆ 3. Tabla Clientes

Guarda los compradores.

id_cliente	nombre	ciudad
1	Juan Pérez	Medellín
2	Laura Díaz	Bogotá

Cada cliente tiene un identificador único.

◆ 4. Tabla Ventas

Aquí se registran las compras realizadas.

id_venta	id_cliente	id_producto	cantidad
1	1	1	10
2	2	2	5

Esta tabla es la más importante porque conecta todo.

🔗 ¿Cómo se relacionan las tablas?

📌 Relación 1

Agricultores → Productos

Un agricultor produce muchos productos

Pedro puede producir varios tipos de aguacates.

📌 Relación 2

Clientes → Ventas

Un cliente puede realizar muchas compras

Juan puede comprar varias veces.

📌 Relación 3

Productos → Ventas

Un producto puede aparecer en muchas ventas

El Aguacate Hass puede venderse muchas veces.

🎯 Explicación Humanizada

Piensa en una tienda real:

👨‍🌾 El agricultor cultiva el aguacate.

🥑 El producto es el aguacate.

👤 El cliente lo compra.

📄 La venta registra la compra.

La base de datos conecta toda esa historia automáticamente.

🧠 Relación visual



Código SQL Completo

```
-- SQL

-- TABLA AGRICULTORES
CREATE TABLE agricultores (
  id_agricultor INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(100),
  finca VARCHAR(100)
);

-- TABLA PRODUCTOS
CREATE TABLE productos (
  id_producto INT PRIMARY KEY AUTO_INCREMENT,
  nombre_producto VARCHAR(100),
  precio DECIMAL(10,2),
  id_agricultor INT,

  FOREIGN KEY (id_agricultor)
  REFERENCES agricultores(id_agricultor)
);

-- TABLA CLIENTES
CREATE TABLE clientes (
  id_cliente INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(100),
  ciudad VARCHAR(100)
);

-- TABLA VENTAS
CREATE TABLE ventas (
  id_venta INT PRIMARY KEY AUTO_INCREMENT,
  id_cliente INT,
  id_producto INT,
  cantidad INT,

  FOREIGN KEY (id_cliente)
  REFERENCES clientes(id_cliente),

  FOREIGN KEY (id_producto)
  REFERENCES productos(id_producto)
);
```

¿Qué aprende un estudiante con este ejemplo?

- ✓ Cómo conectar tablas
- ✓ Cómo usar claves primarias
- ✓ Cómo usar claves foráneas
- ✓ Cómo organizar información real
- ✓ Cómo funciona una base de datos empresarial

Resultado Final

Gracias a las relaciones:

- Sabemos qué agricultor produjo el aguacate.
- Sabemos qué cliente lo compró.
- Sabemos cuántos productos se vendieron.
- Todo queda organizado y profesional.

Así funcionan los sistemas reales de supermercados, empresas y aplicaciones modernas.